

FlowSpace

Interview Prep & Deep-Dive Guide

This document teaches you the *reasoning* behind every architectural decision in FlowSpace. Each section is structured as interviewer questions + confident answers, with key insights, gotchas, and buzzwords to use.

Study Guide v1.0

For: Mohamed El Refaiy

Project: FlowSpace v1.2.0

Date: March 2026

Format: Q&A + Teaching Notes + Gotchas

How to Use This Guide

Each section follows this pattern:

1. **Context** — What the interviewer is really asking and why
2. **Q&A blocks** — Exact questions you might hear, with confident answers
3. **Key Insights** (green) — The deeper understanding that separates you from other candidates
4. **Interview Gotchas** (orange) — Follow-up traps and how to handle them
5. **Keywords** (purple) — Technical terms that signal expertise

Sections

1. The 30-Second Pitch (What Is FlowSpace?)
2. Architecture: Why a Monolith?
3. Authentication: Why gws CLI Instead of Standard OAuth?
4. AI Agent: Why Tool-Calling Over RAG?
5. Streaming: Why SSE Over WebSockets?
6. Caching: Why In-Memory Over Redis/DB?
7. Frontend: Why React Context Over Redux/Zustand?
8. Desktop: Why Tauri Over Electron?
9. Approval Workflow: Why Human-in-the-Loop?
10. State & Storage: Why localStorage Over a Database?
11. The Tool System: Design & Extensibility
12. Error Handling & Resilience
13. Security Deep-Dive
14. Testing Strategy
15. Deployment & Distribution
16. Trade-offs You Consciously Made
17. What You Would Do Differently (Self-Awareness Questions)
18. System Design Whiteboard Walkthrough

1 The 30-Second Pitch

Every interview starts here. You need to explain FlowSpace clearly in under a minute.

Q: Tell me about a project you've built recently.

A: I built FlowSpace, a personal Google Workspace dashboard that consolidates Drive, Gmail, Calendar, and Tasks into a single AI-augmented interface. The core differentiator is an embedded conversational agent that can both *read from* and *write to* all connected Google services using a tool-calling loop. It ships as a web app, a Docker container, and a native macOS desktop app via Tauri v2. The stack is TypeScript end-to-end: React 19 frontend, Express API server, and a Rust shell for the desktop build.

Q: What problem does it solve?

A: Knowledge workers context-switch between 4-5 Google apps dozens of times a day. FlowSpace eliminates that by giving you one view with AI-generated briefings, email triage, calendar prep, and a chat agent that can take action on your behalf — like drafting emails, creating events, or organizing your inbox — all with an approval step so you stay in control.

Q: What's the scale of this project?

A: About 12,000 lines of TypeScript across server and client, 23 AI tool definitions, 20+ API endpoints, 34 React components, and a 112-line Rust shell for the desktop app. It's a single-developer project, which influenced many of the architectural choices — I optimized for velocity and simplicity over horizontal scalability.

Key Insight: Always frame your project in terms of the *problem it solves*, not just the tech. Interviewers want to see product thinking. "I built it because I was frustrated by X" is more compelling than "I wanted to try React 19."

Keywords to use: AI-augmented productivity, tool-calling agent, full-stack TypeScript, monolith-first architecture, native desktop via Tauri, human-in-the-loop approval workflow

2 Architecture: Why a Monolith?

This is the #1 architecture question you'll face. Interviewers expect you to justify *not* using microservices.

Q: Why did you choose a monolithic architecture instead of microservices?

A: Three reasons. First, **team size**: this is a single-developer project. Microservices add operational overhead (service discovery, inter-service auth, distributed tracing) that only pays off with multiple teams working independently. Second, **shared context**: the API server, auth layer, and AI chat all need access to the same Google OAuth credentials and in-memory cache — splitting them into separate services would mean duplicating auth state or adding a shared session store. Third, **deployment simplicity**: a single Express process can serve both the API and the frontend, which makes the Tauri desktop bundle trivial — I just spawn one Node process and point the WebView at it.

Q: When would you break it apart?

A: If the AI chat workload needed to scale independently from the dashboard reads — for example, if chat requests were CPU-bound and starving the simple GET endpoints. I'd extract the `/api/chat` SSE endpoint into its own service first, since it's the only long-lived connection. The REST endpoints are stateless and could move behind a gateway later. I'd also split if multiple developers needed to ship different parts on different cadences.

Key Insight: The "monolith vs microservices" question is really about **understanding trade-offs**. Never say "monolith bad" or "microservices good." The right answer is always "it depends on team size, deployment requirements, and where the complexity boundaries are." For a single-developer project with shared state, a monolith is the *correct* engineering decision, not a shortcut.

Interview Gotcha: "But doesn't a monolith mean you can't scale?" No. A monolith can scale vertically (bigger machine) and horizontally (multiple instances behind a load balancer) as long as shared state is externalized. FlowSpace's state is per-user (filesystem + localStorage), so horizontal scaling is straightforward if needed.

Keywords to use: monolith-first, shared-nothing architecture, vertical scaling, deployment unit, service boundary, Conway's Law, operational overhead

3 Authentication: Why gws CLI Instead of Standard OAuth?

This question tests whether you understand OAuth deeply enough to know when to deviate from the standard flow.

Q: Why didn't you use a standard OAuth2 web flow?

A: A standard OAuth2 web flow requires the user to create a Google Cloud project, enable APIs, configure an OAuth consent screen, and download a client secret — that's a 15-minute setup before they can even try the app. Since FlowSpace is open-source and designed for personal use, I wanted **zero-configuration onboarding**. The gws CLI already handles the OAuth dance with its own registered client, so users just click "Sign in with Google," authorize in the browser, and FlowSpace imports the credentials. The trade-off is a runtime dependency on the gws CLI binary, but I bundle it in the Tauri desktop build and auto-install it in the web version.

Q: Walk me through the credential flow. Where are tokens stored?

A: The flow has four steps. First, we check if gws CLI is installed (`/api/auth/gws-status`). If not, we install it via npm. Then we trigger `gws auth login` , which opens the browser for Google consent. After authorization, we call `gws auth export --unmasked` to get the access token, refresh token, client ID, and client secret as JSON. We save this to a per-account credential file in the app's data directory — `~/Library/Application Support/FlowSpace/.gws-credentials.{email}.json` . The access token is short-lived (1 hour); we refresh it using the refresh token and client credentials via Google's token endpoint. Critically, tokens never touch the frontend — they stay server-side, and the gws CLI receives tokens via environment variables, never command-line arguments (which would be visible in `ps` output).

Q: How do you handle multiple Google accounts?

A: There's an `.accounts.json` manifest that lists all connected accounts with their email, display name, and a pointer to their credential file. The server maintains an "active account" concept — all API calls use the active account's OAuth client. Switching accounts is a POST to `/api/accounts/switch` that changes the active account ID and clears the in-memory cache so data refreshes. On the frontend, conversations are scoped per account in localStorage using the account key as a namespace.

Key Insight: The credential hierarchy (active account → legacy single-account → OAuth2 web tokens → Application Default Credentials) is a pattern called **fallback chain**. It ensures backward compatibility as the auth system evolved: users who authenticated before multi-account was added still work, and developers can use their gcloud ADC for local testing. This is a real-world migration strategy — never break existing users when adding new features.

Interview Gotcha: "Isn't it insecure to store credentials on disk?" All major tools do this — gcloud, aws-cli, kubectl, Docker. The credentials are in the user's home directory with standard filesystem permissions. The alternative (a remote token store) would require its own authentication, creating a chicken-and-egg problem. For a local-first app, filesystem storage with proper permissions is the standard approach.

Keywords to use: OAuth2 authorization code flow, refresh token rotation, credential delegation, fallback chain, zero-configuration onboarding, token hygiene, principle of least privilege

4 AI Agent: Why Tool-Calling Over RAG?

This is your chance to show you understand modern AI application patterns deeply.

Q: How does the AI agent work?

A: It uses a **tool-calling loop** pattern. The user sends a message, we append it to the conversation history along with a system prompt, and send it to the LLM (GLM-4.7 via Z.AI's OpenAI-compatible API). The LLM can either respond with text or request one or more tool calls. If it requests tools, we execute them, append the results to the conversation, and send it back to the LLM. This loops up to 5 times per turn. The LLM decides which tools to call based on the user's intent — it has 23 tools covering read, write, and workflow operations across all Google services.

Q: Why tool-calling instead of RAG?

A: RAG (Retrieval-Augmented Generation) is for when you have a static knowledge base that you want the LLM to reason over. FlowSpace's data is *live and dynamic* — your unread emails change every minute, your calendar changes throughout the day. There's nothing to pre-embed. Instead, tools let the LLM make real-time API calls: "search_emails for budget reports" returns the actual current emails. And critically, FlowSpace needs to *take actions*, not just retrieve information — sending emails, creating events, archiving threads. RAG can't do that. Tool-calling is the right pattern when you need real-time data access and write operations.

Q: Why GLM-4.7? Why not GPT-4 or Claude?

A: Two reasons: cost and configurability. GLM-4.7 provides strong tool-calling capability at a fraction of the cost of GPT-4. For a personal productivity tool making dozens of API calls per session, cost matters. But I designed the system to be **provider-agnostic** — the LLM client uses the OpenAI SDK's compatibility mode, and the Settings page lets users configure any OpenAI-compatible provider (OpenAI, Anthropic via proxy, local models via Ollama). The model choice is a runtime configuration, not an architectural coupling.

Q: Why cap it at 5 tool rounds?

A: Safety and cost. Each round involves an LLM call (which costs tokens) plus one or more Google API calls (which have rate limits). Without a cap, a confused model could loop indefinitely — calling search, not finding results, rephrasing, searching again. Five rounds is enough for complex multi-step workflows (e.g., "find my meeting with Sarah, get the agenda doc, summarize it") while preventing runaway execution. If the model hasn't resolved the task in 5 rounds, it's better to ask the user to clarify than to keep guessing.

Key Insight: The tool-calling pattern is sometimes called "ReAct" (Reasoning + Acting). The key insight is that the LLM acts as an **orchestrator**, not a data store. It reasons about what information it needs, calls tools to get it, reasons about the results, and decides the next action. Your tools are the LLM's "hands" — they should be atomic, well-documented, and predictable. The system prompt is the LLM's "training manual" for how to use those hands.

Interview Gotcha: "What if the LLM hallucinates a tool call?" Every tool execution is wrapped in try/catch with typed error classification (auth_expired, rate_limited, tool_timeout, validation_failed). Invalid arguments fail at the gws CLI level with a clear error, which is fed back to the LLM so it can self-correct. The approval workflow for write tools is the ultimate safety net — even if the LLM generates a wrong email draft, the user sees it before it's sent.

Keywords to use: ReAct pattern, tool-calling loop, function calling, agentic AI, orchestrator pattern, provider-agnostic, OpenAI-compatible API, token budget, graceful degradation

5 Streaming: Why SSE Over WebSockets?

Q: Why Server-Sent Events instead of WebSockets for the chat?

A: Three reasons. First, **directionality**: chat streaming is fundamentally server-to-client. The user sends a message via a regular POST, then the server streams back chunks, tool events, and the final response. SSE is purpose-built for this one-way flow. WebSockets would give me bidirectional communication that I don't need, adding complexity for no benefit. Second, **HTTP compatibility**: SSE works over standard HTTP/1.1, goes through proxies, load balancers, and CDNs without special configuration. WebSockets require an upgrade handshake that some corporate proxies block. Third, **simplicity**: SSE is just `res.write()` on an Express response with `text/event-stream` content type. No library needed. WebSockets need a separate server (`ws`) or Socket.io with its own connection lifecycle.

Q: But SSE is one-way. How does the user approve a pending action?

A: Approval is a separate POST request to `/api/chat/runs/:runId/approve`. This is actually a cleaner separation of concerns: the SSE stream is for *observing* the run (progress events, tool activity, final response), and REST endpoints are for *interacting* with it (approve, dismiss, cancel). Mixing control signals into the data stream would make the protocol harder to reason about.

Key Insight: The rule of thumb: use SSE when data flows one way (server → client), use WebSockets when you need true bidirectional real-time communication (collaborative editing, multiplayer games, live chat between users). Most "real-time" features in web apps are actually one-way broadcasts that SSE handles perfectly — notifications, live feeds, streaming AI responses, stock tickers.

Interview Gotcha: "What about reconnection?" SSE has built-in reconnection with `Last-Event-ID`. If the connection drops, the browser automatically reconnects and sends the last event ID so the server can resume. WebSockets require you to implement this yourself. This is another reason SSE wins for streaming use cases.

Keywords to use: Server-Sent Events, unidirectional streaming, text/event-stream, HTTP/1.1 compatibility, automatic reconnection, separation of concerns (observation vs. interaction)

6 Caching: Why In-Memory Over Redis/DB?

Q: You're using a simple in-memory cache with 60-second TTL. Why not Redis?

A: This is a **single-user, single-process** application. Redis solves the problem of sharing cache across multiple server instances — but FlowSpace runs one Express process per user (or per desktop app). There's no cache to share. Adding Redis would mean adding an external dependency (install, configure, monitor) for zero architectural benefit. The in-memory Map with TTL is ~15 lines of code, has zero latency (no network hop), and is perfectly reliable for a single-process app.

Q: What happens when the server restarts? You lose the cache.

A: Yes, and that's fine. The cache exists purely to avoid hammering Google APIs with repeated identical requests within a short window (like when the dashboard loads and fetches stats, recent files, emails, and events nearly simultaneously). A cold cache just means the first request after restart takes slightly longer. There's no critical state in the cache — it's all derivable from Google APIs. If I needed persistent caching, I'd use SQLite (which is already a dependency) rather than Redis, keeping the deployment footprint small.

Q: Why 60 seconds? Why not longer?

A: It's a balance between API rate limit protection and data freshness. Google APIs have per-user rate limits (typically 10 requests/second for Gmail). A 60-second TTL means rapid page navigation won't trigger redundant API calls. But 60 seconds is short enough that when a user archives an email or creates an event, they'll see the updated state on their next natural interaction. The briefing endpoint uses a 10-minute TTL because it requires an LLM call, which is expensive and the content (daily summary) doesn't change often.

Key Insight: Cache design always answers three questions: **What** to cache (derived, expensive-to-compute data), **When** to invalidate (TTL for time-based, event-based for mutations), and **Where** to store it (in-process for single instance, distributed for multi-instance). The simplest correct answer wins. Don't add infrastructure complexity to solve a problem you don't have.

Keywords to use: TTL-based eviction, cache-aside pattern, cold start, rate limit protection, data freshness vs. API cost trade-off, YAGNI principle

7 Frontend: Why React Context Over Redux/Zustand?

Q: Why React Context for state management instead of Redux or Zustand?

A: The state in FlowSpace is **colocated with a single concern**: the chat experience. ChatContext manages conversations, messages, approvals, and thread groups. It's one context provider wrapping the app. Redux shines when you have *many independent state slices* that need to be composed and when multiple unrelated components need to subscribe to specific slices without re-rendering everything. FlowSpace's state is more like a tree — conversation → messages → tool events — which maps naturally to a single context with `useContext`. Adding Redux would mean boilerplate (actions, reducers, selectors) for a state shape that doesn't benefit from it.

Q: Don't you get performance issues with Context re-renders?

A: Yes, that's a real concern. When any value in the context changes, all consumers re-render. I mitigate this in two ways: first, the context value is memoized, so only meaningful state changes trigger re-renders. Second, heavy rendering components like ChatThread receive individual messages as props, not the full conversation — so React's reconciliation only updates the changed message, not the entire thread. For a single-user app with modest component counts, this is performant enough. If I saw render bottlenecks in profiling, I'd reach for Zustand (lighter than Redux, selector-based subscriptions) before Redux.

Key Insight: The honest answer is: **React Context is sufficient for single-concern state in a medium-complexity app**. The "you need Redux" advice comes from large team environments where state is spread across dozens of domains. For a personal project with one state tree, Context + `useMemo` + prop drilling where appropriate is cleaner and faster to develop. Know when to reach for more — that's what interviewers are testing.

Keywords to use: Context API, colocated state, render optimization, memoization, selector-based subscriptions, prop drilling, state colocation principle

8 Desktop: Why Tauri Over Electron?

Q: Why Tauri v2 instead of Electron for the desktop app?

A: Bundle size and resource usage. Electron ships an entire Chromium browser (~150 MB) plus a Node.js runtime. Tauri uses the system's native WebView (WebKit on macOS), so the Rust binary itself is tiny. The FlowSpace desktop app is ~200 MB total, and most of that is the bundled Node.js and server code, not the Tauri shell. An equivalent Electron build would be 400-500 MB. Memory usage is also lower — Tauri doesn't spawn a separate browser process. For a personal tool that runs all day, resource efficiency matters.

Q: But you're still bundling Node.js. Doesn't that negate the size advantage?

A: Partially, yes. The Node.js binary is ~80 MB. But here's the key architectural decision: the Express server is the *source of truth* for data, auth, and AI chat. I could have rewritten the API layer in Rust, but that would mean maintaining two implementations of the same logic. By spawning Node as a child process, I get 100% code sharing between web and desktop — same server, same API, same behavior, zero divergence risk. The Tauri shell is just 112 lines of Rust that manages the process lifecycle. If Node bundle size became a real problem, I could explore Bun (smaller binary) or a Rust rewrite of critical paths.

Q: How does the Tauri app manage the server lifecycle?

A: The Rust code in `lib.rs` does four things: (1) sets environment variables including `FLOWSPACE_PRODUCTION=1` and augments PATH to find Node on macOS, (2) spawns `node server.mjs` as a child process using Tauri's shell plugin, (3) polls TCP port 3000 every 500ms up to 60 times waiting for the server to respond, and (4) navigates the WebView to `http://localhost:3000` once it's ready. On window close, it kills the server process. The server process handle is stored in a `Mutex<Option<CommandChild>>` app state for safe concurrent access.

Key Insight: The decision to spawn Node instead of rewriting in Rust is a **build vs. buy decision at the module level**. The cost of code duplication (two implementations of the same API) is almost always higher than the cost of a larger binary. Ship the same code everywhere; optimize later if size becomes a measured problem, not a hypothetical one.

Interview Gotcha: "What about cross-platform?" Tauri v2 supports macOS, Windows, and Linux. Currently FlowSpace only builds for macOS because that's the primary target audience, but the Tauri layer is cross-platform by design. The Node.js server code is already platform-agnostic.

Keywords to use: native WebView, system webview vs. bundled Chromium, child process management, code sharing across deployment targets, process lifecycle management, sidecar pattern

9 Approval Workflow: Why Human-in-the-Loop?

Q: Why require user approval for write operations?

A: Trust calibration. An AI agent that can send emails on your behalf is powerful but dangerous. One hallucinated email to your CEO could be career-ending. The approval workflow is a *trust boundary*: read operations (search, list, read) execute automatically because the worst case is showing wrong data that the user can verify visually. Write operations (send email, create event, modify spreadsheet) require approval because they have *real-world side effects* that may be irreversible. This is the same principle behind Unix's `rm -i` flag or Git's pre-push hooks — add friction where mistakes are costly.

Q: How does the approval UI work technically?

A: When the LLM requests a write tool, the server calls `buildApprovalRequest()` which creates a structured form with the tool's arguments as editable fields. For example, a `send_email` approval shows To, Subject, and Body fields that the user can review and modify before confirming. The SSE stream emits a `tool_event` with status `approval_required`, and the run transitions to `awaiting_approval`. The frontend renders an approval card in the chat thread. When the user clicks Approve, a POST to `/api/chat/runs/:runId/approve` triggers actual execution with the (potentially edited) arguments. The key insight is that the user can *edit* the AI's proposed action, not just accept/reject it.

Key Insight: Human-in-the-loop is not just a safety feature — it's a **UX pattern that builds trust incrementally**. Users start by reviewing every action. Over time, as they see the AI making good decisions, they approve faster. Eventually, you could add "auto-approve for this tool" as a preference. But starting with mandatory approval means you never have to apologize for an AI sending a wrong email. Trust is earned, not assumed.

Keywords to use: human-in-the-loop, trust boundary, side-effect isolation, read/write separation, approval workflow, editable action preview, progressive trust

10 State & Storage: Why localStorage Over a Database?

Q: Why store conversations in localStorage instead of a database?

A: Privacy and simplicity. Conversations contain the full text of your emails, calendar events, and whatever you discuss with the AI. Storing this in a cloud database would create a privacy liability — a data breach would expose all of a user's Google Workspace content. localStorage keeps everything on the user's machine. It's also zero-configuration: no database to set up, no migrations to run, no connection strings. For a personal tool, this is the right trade-off.

Q: What about the 5-10 MB localStorage limit?

A: It's a real constraint. A heavy user with hundreds of conversations could hit it. I mitigate this with time-based grouping (conversations are grouped as Today / Yesterday / This Week / Older) and the UI naturally encourages archiving. If this became a problem, I'd migrate to IndexedDB (50+ MB limit, same privacy model) or the SQLite backend via better-sqlite3 which is already a dependency. The migration path is straightforward because conversations are JSON-serializable.

Q: What about server-side state? What's stored on the filesystem?

A: The server stores five types of JSON files in the data directory: **credentials** (OAuth tokens per account), **accounts manifest** (list of connected accounts), **memories** (long-term AI context per user), **dynamic tools** (user-created tool definitions), and **follow-up state** (commitment tracker). Each is a single JSON file that's read on demand and written on change. For a single-user app, this is simpler and more reliable than a database — no migrations, no schema versions, no corruption risk from improper shutdowns. The data is also trivially portable — copy the directory to back up or migrate everything.

Key Insight: The storage decision maps directly to the **data's lifecycle and sensitivity**. Ephemeral data (cache) → memory. Session data (conversations) → client-side storage. Persistent config (credentials, preferences) → filesystem. Shared data (none, single-user) → database. FlowSpace has no shared data, so no database is needed. This is a feature, not a limitation.

Keywords to use: local-first architecture, data sovereignty, zero-configuration storage, JSON-based persistence, IndexedDB migration path, portable data directory

11 The Tool System: Design & Extensibility

Q: How did you design the tool system? How are 23 tools organized?

A: Tools are organized into four tiers by complexity and risk. **Read tools** (14) query Google APIs and return structured data — they're safe to auto-execute. **Write tools** (9) modify user data and require approval. **Bulk inbox actions** (8) operate on multiple items and show a preview before execution. **Workflow tools** (4) compose multiple reads into higher-level operations like standup reports or meeting prep. Each tool is defined as a JSON schema (name, description, parameters) that the LLM sees, plus an execution function that maps to gws CLI commands or direct Google API calls.

Q: How do tools execute? Why subprocess instead of direct API calls?

A: Most tools execute via the **gws CLI as a subprocess** using Node's `execFile` (not `exec`). This is intentional for three reasons: (1) **Security** — `execFile` doesn't spawn a shell, preventing command injection. The token is passed via environment variable, not command-line argument. (2) **Isolation** — each tool call runs in its own process with its own timeout (30s). A hung API call doesn't block the server. (3) **Abstraction** — the gws CLI handles the complexity of Google API pagination, field formatting, and error messages. My tools are thin wrappers that parse the CLI output.

Q: Tell me about the dynamic tool composition feature.

A: Users can create custom tools by chaining existing tools into multi-step workflows. A dynamic tool definition is an array of `ToolStep` objects, each with an action (existing tool name), arguments (with template variables), and an optional output key for passing results between steps. For example, a "log expense" tool could read a spreadsheet to find the next row, then append a new row with the expense data. These are stored in `.dynamic-tools.json` and registered at runtime. It's essentially a simple workflow engine built on top of the existing tool infrastructure.

Key Insight: The tool system follows the **Unix philosophy**: each tool does one thing well, and complex behavior emerges from composition. The 23 tools are atomic building blocks. The LLM composes them at runtime (multi-round tool calling), and users can pre-compose them into reusable

workflows (dynamic tools). This is more maintainable than monolithic "smart" tools that try to handle every case internally.

Keywords to use: Unix philosophy, tool composition, subprocess isolation, command injection prevention, execFile vs exec, workflow engine, schema-driven tool definitions

12 Error Handling & Resilience

Q: How do you handle errors in the tool-calling loop?

A: Errors are classified into typed categories: `auth_expired`, `rate_limited`, `tool_timeout`, `validation_failed`, and `unknown`. Each has a different recovery strategy. Auth errors stop the run and prompt re-authentication. Rate limit errors are reported to the LLM so it can pace itself. Timeout errors (30s) abort the tool and let the LLM try an alternative approach. Validation errors (bad arguments) are fed back to the LLM for self-correction. Unknown errors are logged with full context but only a sanitized message is sent to the frontend — no stack traces, no internal paths.

Q: What if the LLM returns malformed tool calls?

A: The tool execution layer validates the tool name against the registered tool set and the arguments against the expected schema. If the LLM requests a non-existent tool, we return an error result that says "tool not found" with the list of available tools. If arguments are missing or wrong type, the gws CLI rejects them with a descriptive error. In both cases, the error is appended to the conversation so the LLM can self-correct in the next round. The 5-round limit ensures this correction loop terminates.

Key Insight: Error classification is more important than error handling. Generic try/catch with a "something went wrong" message is useless. By classifying errors into actionable categories, each layer of the system can make intelligent decisions: the tool layer retries or fails fast, the chat loop decides whether to continue or stop, and the frontend shows the user a relevant message ("your session expired" vs. "we hit Google's rate limit, try again in a minute").

Keywords to use: typed error classification, self-correcting agents, graceful degradation, error boundary, fail-fast, sanitized error messages, defense in depth

13 Security Deep-Dive

Q: What are the main attack surfaces, and how do you mitigate them?

A: Four attack surfaces. **(1) Token theft:** Credentials are stored in the user's home directory with standard file permissions. Tokens are passed to subprocesses via env vars, never CLI args (which appear in `ps` output). Refresh tokens are server-side only. **(2) Command injection:** All subprocess execution uses `execFile` which doesn't spawn a shell. Arguments are passed as an array, not a concatenated string. **(3) Prompt injection:** The LLM processes user messages that could contain adversarial prompts. The approval workflow is the safety net — even if the LLM is tricked into calling `send_email` with malicious content, the user sees and approves it first. **(4) Data exfiltration via LLM:** User data is sent to the LLM provider (Z.AI) for processing. This is disclosed and mitigated by making the LLM provider configurable — users can point to a local model via Ollama for full privacy.

Q: How do you prevent the AI from taking destructive actions?

A: Three layers. **Layer 1:** Read/write tool separation — the LLM can freely read, but all writes require approval. **Layer 2:** Approval UI with editable fields — the user sees exactly what will happen and can modify it. **Layer 3:** Undo capability — bulk inbox actions have an audit log, and the `restore_email_threads` tool can reverse archive/trash operations. This defense-in-depth means a single failure (LLM hallucination, user misclick, UI bug) doesn't cause irreversible damage.

Key Insight: In AI-powered applications, the **trust boundary** is between the LLM's output and the system's actions. Never trust LLM output as validated input. Treat every tool call from the LLM the same way you'd treat user input in a web form — validate, sanitize, and require confirmation for dangerous operations. The approval workflow is essentially a CSRF protection mechanism for AI actions.

Keywords to use: defense in depth, trust boundary, principle of least privilege, command injection prevention, prompt injection mitigation, STRIDE threat model, audit trail

14 Testing Strategy

Q: How do you test an AI-powered application?

A: I test in layers. **Unit tests** (Vitest) cover pure logic: message parsing, suggestion extraction, triage heuristics, credential resolution. These are deterministic and fast. **Integration tests** cover API endpoints with mocked Google API responses — verifying that the server correctly handles auth, caching, and error cases. **The AI layer is deliberately not unit-tested** for LLM output (which is non-deterministic), but the *infrastructure around it* is tested: tool registration, approval request building, error classification, SSE event formatting. The tool execution functions are tested with fixture data from real gws CLI output.

Q: Why not test the LLM responses?

A: LLM output is non-deterministic by design — the same prompt can produce different responses. Testing "did the model say the right thing" is fragile and expensive (requires API calls). Instead, I test the *contract*: "given a tool call from the LLM, does the system execute it correctly?" and "given a tool result, does the system format it correctly for the frontend?" The LLM is treated as an external service with well-defined inputs (system prompt + tools + messages) and outputs (text or tool calls). Everything around that boundary is testable and tested.

Key Insight: When testing AI applications, draw a line around the non-deterministic part (LLM inference) and test everything else. Test the **plumbing, not the prediction**. Verify tool schemas are valid, approval requests have all required fields, error handling works, streaming events are well-formed. This gives you confidence that when the LLM behaves correctly, the system handles it correctly — and when the LLM misbehaves, the system handles that correctly too.

Keywords to use: contract testing, deterministic vs. non-deterministic testing, test pyramid, fixture-based testing, LLM evaluation (evals), boundary testing

15 Deployment & Distribution

Q: Walk me through your deployment strategy.

A: FlowSpace has three deployment targets. **Local development:** `make dev` runs Tailwind in watch mode + tsx server with Vite HMR middleware. **Docker:** Two-stage build — stage 1 installs deps and builds the frontend, stage 2 copies only production files and deps. Final image is ~500 MB on Node 22 Alpine. **macOS desktop:** `make tauri-build` runs a pre-build script that bundles Node.js and gws CLI as sidecars, then Tauri builds the .app and .dmg with code signing and notarization for distribution.

Q: Why a two-stage Docker build?

A: Stage 1 needs build tools (python3, make, g++) for native Node modules like better-sqlite3 and sharp. Stage 2 only needs the Node runtime and production dependencies. By splitting stages, the build tools (~200 MB) don't end up in the production image. This is a standard Docker best practice for Node.js apps with native addons. The final image only contains what's needed to run, not what's needed to build.

Key Insight: Multi-stage Docker builds are one of the most impactful DevOps optimizations. They let you use a rich build environment (compilers, dev headers) without paying for it in production image size. Always separate "build" from "run" concerns in your Dockerfile.

Keywords to use: multi-stage Docker build, sidecar bundling, code signing and notarization, HMR (Hot Module Replacement), production-grade build pipeline

16 Trade-offs You Consciously Made

This is the section that separates senior engineers from junior ones. Every decision has a cost. Owning those costs shows maturity.

DECISION	WHAT YOU GAINED	WHAT YOU GAVE UP
Express monolith	Simple deployment, shared auth, single process, fast iteration	Independent scaling of API vs. chat, microservice-level isolation
gws CLI for auth	Zero-config onboarding, no GCP project needed	Runtime dependency on external binary, less control over OAuth scopes
In-memory cache	Zero latency, no infrastructure, 15 lines of code	Lost on restart, no cross-instance sharing, 60s eventual consistency
SSE over WebSocket	Simple, proxy-friendly, built-in reconnect, standard HTTP	Unidirectional only, separate POST needed for user actions
Tauri over Electron	Smaller bundle (~200 MB vs ~500 MB), lower memory usage	macOS-only WebView quirks, smaller ecosystem
localStorage for chat	Privacy-first, zero-config, instant persistence	5-10 MB limit, no cross-device sync, no search across conversations
Mandatory approval	User safety, trust building, catch AI mistakes	Friction for power users, can't fully automate workflows
GLM-4.7 default	Cost-effective, good tool-calling support	Less capable reasoning than GPT-4/Claude for edge cases
Subprocess for tools	Isolation, timeout control, security (no shell)	Process spawn overhead (~50ms), stdout parsing required
No database	Zero ops, portable data dir, no migrations	No complex queries, no relations, manual JSON management

Key Insight: When an interviewer asks "what would you do differently?", they're testing self-awareness. The best answer format is: "I chose X because of constraint Y. The cost was Z. If that constraint changed (e.g., multi-user, larger team, higher scale), I'd migrate to W." Never say "I'd do everything the same" — that signals inexperience.

17 What You Would Do Differently

Prepare for these "retrospective" questions. They test engineering judgment, not just skill.

Q: If you started over, what would you change?

A: Three things. **First, I'd use a typed RPC framework like tRPC** instead of hand-written Express routes and a manual API client. FlowSpace has 20+ endpoints, each with its own request/response types defined separately on server and client. tRPC would give me end-to-end type safety with zero code generation. **Second, I'd use Zustand instead of React Context** from the start. The ChatContext grew to 31 KB because it handles too many concerns. Zustand's slice pattern would keep state modular without the re-render overhead. **Third, I'd add structured logging from day one.** Currently the server uses `console.log`. In production debugging, structured JSON logs with correlation IDs (linking a chat request to its tool calls) would save hours.

Q: What's the biggest technical debt?

A: The `server.ts` file at 3,633 lines. It handles auth, caching, all API routes, chat orchestration, and Vite middleware in a single file. It should be split into modules: `routes/auth.ts`, `routes/workspace.ts`, `routes/chat.ts`, `middleware/cache.ts`, `services/google.ts`. The file grew organically during rapid prototyping, and I've been shipping features faster than refactoring. This is a conscious trade-off — for a solo project, shipping beats perfection — but it would be the first thing I'd refactor before onboarding another developer.

Q: How would you scale this to 10,000 users?

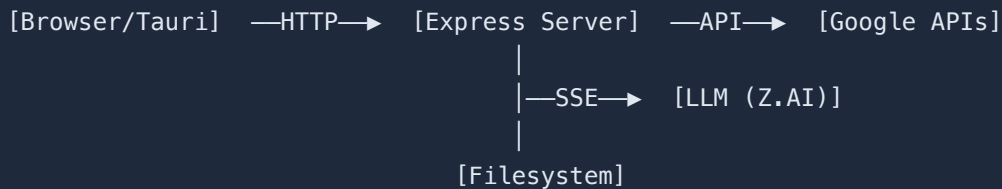
A: FlowSpace is designed as a *local-first* tool — each user runs their own instance. But if I were building a SaaS version: (1) Extract auth into a session service backed by a database (PostgreSQL + Redis sessions). (2) Move the cache to Redis for cross-instance sharing. (3) Put the chat endpoint behind a queue (like BullMQ) so LLM calls don't block server workers. (4) Store conversations in a database instead of localStorage, with end-to-end encryption. (5) Add a connection pooler for Google API calls to respect per-user rate limits without affecting other users. The monolith can stay as one deployable — the bottleneck is the LLM API and Google API rate limits, not the server architecture.

Key Insight: Interviewers love the "scale to X users" question because it tests whether you understand **where the bottlenecks actually are**, not whether you can list "add Redis, add Kubernetes." Always identify the actual bottleneck first (LLM API latency, Google rate limits), then describe targeted solutions for those bottlenecks. Generic "just add more servers" answers signal you don't understand the problem.

18 System Design Whiteboard Walkthrough

If asked to whiteboard FlowSpace, draw this in order. Practice drawing it in under 5 minutes.

Step 1: High-Level Boxes (30 seconds)



Say: "Single Express server serves both the API and frontend. Talks to Google for data, LLM for AI, filesystem for persistence."

Step 2: Auth Flow (1 minute)

```

User → "Sign In" → gws CLI → Google OAuth → Token saved to filesystem
      → Server loads token → Creates Google API clients → Ready
  
```

Say: "Auth is delegated to the gws CLI for zero-config onboarding. Tokens stored in the user's home directory. Credential hierarchy with fallback chain for backward compatibility."

Step 3: Chat Flow (2 minutes)

```

User message → POST /api/chat
  → Build system prompt (timezone, persona, memories)
  → Send to LLM
  → LLM returns tool_calls[]
    → Read tool? Execute immediately, feed result back
    → Write tool? Build approval request, pause, wait for user
  → Loop up to 5 rounds
  → Stream response via SSE (chunks + tool events + final payload)
  → Extract memories for next time
  
```

Say: "Tool-calling loop with human-in-the-loop for writes. Max 5 rounds to prevent runaway execution. Streaming via SSE for real-time UX."

Step 4: Data & Cache (1 minute)

```
API Request → Check in-memory cache (60s TTL)
  → HIT: Return cached
  → MISS: Call Google API → Cache result → Return

Conversations: localStorage (client-side, private)
Credentials: Filesystem (server-side, per-account)
Memories: Filesystem (server-side, per-user hash)
```

Say: "Read-through cache for Google APIs. All persistent state is local — no cloud database needed for a personal tool."

Step 5: Deployment (30 seconds)

```
Web: Express on port 3000
Desktop: Tauri spawns Node as child process, WebView → localhost:3000
Docker: Multi-stage build, Node 22 Alpine
```

Say: "Same codebase, three deployment targets. Tauri uses the same Express server as web — zero code divergence."

Key Insight: When whiteboarding, **narrate your trade-offs as you draw**. Don't just draw boxes — say "I chose this because..." at each decision point. This shows the interviewer that every box is intentional, not accidental. The goal isn't a perfect diagram; it's demonstrating that you can reason about systems end-to-end.

* Rapid-Fire Review Cards

Quick-reference answers for common follow-ups. Read these the morning of your interview.

QUESTION	ONE-SENTENCE ANSWER
Why TypeScript everywhere?	End-to-end type safety between server and client, shared type definitions, fewer runtime errors.
Why Vite over Webpack?	10x faster HMR via native ES modules, simpler config, built-in TypeScript support.
Why Tailwind CSS v4?	Utility-first = faster iteration, no CSS naming debates, v4 uses CSS custom properties natively.
Why Framer Motion?	Declarative animation API that integrates cleanly with React's component model.
Why no React Router?	Single-page app with tab-based navigation — view switching via state is simpler than URL routing.
Why OpenAI SDK for non-OpenAI model?	OpenAI's API format is the de facto standard. Most providers offer compatible endpoints.
Why <code>execFile</code> not <code>exec</code> ?	<code>execFile</code> skips the shell, preventing command injection. Arguments are an array, not a string.
Why 23 tools, not fewer?	Atomic tools compose better. The LLM chains them at runtime for complex workflows.
Why no WebSocket?	Chat streaming is unidirectional (server → client). SSE is simpler, proxy-friendly, and has auto-reconnect.
Why no database?	Single-user, local-first app. JSON files in home directory = zero ops, full portability, privacy.
Why Rust for desktop?	Tauri uses the system WebView (no bundled Chromium). Rust shell is 112 lines. 60% smaller than Electron.
How do you prevent AI mistakes?	Human-in-the-loop approval for all write operations. User sees and can edit before executing.
What's the biggest tech debt?	server.ts at 3,633 lines. Should be split into route modules and service layers.

How would you add auth for SaaS?	PostgreSQL for accounts, Redis for sessions, JWT or session cookies, per-user token encryption.
What would break at scale?	In-memory cache (needs Redis), filesystem storage (needs DB), LLM latency (needs queue).

FlowSpace Interview Prep Guide — v1.0 — March 2026

Study this, practice the whiteboard walkthrough, and own your trade-offs.

You built this. You know why every line exists. That's your superpower.